

Chapter 1

Proximity Service

In this chapter we design a proximity service.

- A proximity service is used to discover nearby places such as restaurants, hotels, theaters, museums, etc., and is a core component that powers features like finding the best restaurants nearby on Yelp or finding k-nearest gas stations on Google Maps.
- Figure 1.1. shows the user interface via which you can search for nearby restaurants on Yelp [1]
- Note the map tiles used in this book are from Stamen Design [2] and data are from OpenStreetMap [3]

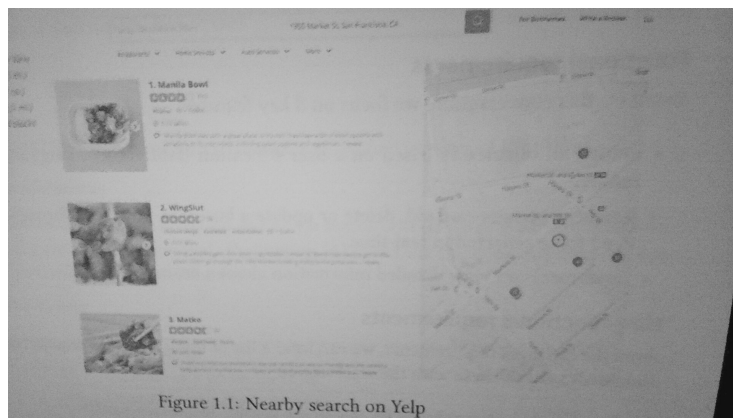


Figure 1.1: Nearby search on Yelp

Nearby search on yelp

Step 1 - Understand the Problem and Establish Design Scope

Yelp supports many features and it is not feasible to design all of them in an interview session, so it's important to narrow down the scope by asking questions

The interactions between the candidate and interviewer could look like this:

- Candidate: Can a user specify the search radius? If there are not enough businesses within the search radius, does the system expand the search?
- Interviewer: That's a great question. Let's assume we only care about businesses within a specified radius. If time allows, we can then discuss how to expand the search if there are not enough businesses within the radius
- Candidate: Can a user change the search radius on the UI?
- Interviewer: Yes, we have the following options: 0.5km (0.31 mile), 1km (0.62 mile), 2km (1.24 mile), 5km (3.1 mile), and 20 km (12.42 mile)
- Candidate: How does business information get added, deleted, or updated? Do we need to reflect these operations in real-time?
- Interviewer: Business owners can add, delete or update a business. Assume we have a business agreement upfront that newly added/updated businesses will be effective the next day.
- Candidate: A user might be moving while using the app/website, so the search results could be slightly different after a while. Do we need to refresh the page to keep the results up to date?
- Interviewer: Let's assume a user's moving speed is slow and we don't need to constantly refresh the page

Functional requirements

Based on this conversation, we focus on 3 key features:

- Return all businesses based on a user's location (latitude and longitude pair) and radius
- Business owners can add, delete, or update a business but this information doesn't need to be reflected in real time
- Customers can view detailed information about a business

Non-functional requirements

From the business requirements, we can infer a list of non-functional requirements. You should also check these with the interviewer.

- Low latency: Users should be able to see nearby businesses quickly

- Data privacy: Location info is sensitive data
 - When we design a location-based service (LBS), we should always take user privacy into consideration
 - We need to comply with data privacy laws like General Data Protection Regulation (GDPR) [4] and California Consumer Privacy Act (CCPA) [5], etc.
 - High availability and scalability requirements: We should ensure our system can handle the spike in traffic during peak hours in densely populated areas

Back-of-the-envelope-estimation

Let's take a look at some back-of-the-envelope-estimation

- Let's take a look at some back-of-the-envelope calculations to determine the potential scale and challenges our solution will need to address
- Assume we have 100 million daily active users and 200 million businesses

Calculate QPS

- Seconds in a day = $24 \times 60 \times 60 = 86400$ seconds. We can round it up to 10^5 for easier calculation
- 10^5 is used in this book for easier calculation and is used throughout this book to represent seconds in a day
- Assume a user makes 5 search queries per day
- Search QPS = $\frac{100 \text{ million} \times 5}{10^5} = 5000$

Table 1.1: Request Parameters

Field	Description	Type
Latitude	Latitude of a given location	decimal
Longitude	Longitude of a given location	decimal
Radius	Optional. Default is 5000 meters	int

```

1  {
2    "total": 18,
3    "businesses": [{business object}]
4  }
```

The business object contains everything needed to render the search result page, but we may still need additional attributes such as pictures, reviews, star rating, etc., to render the business detail page.

Therefore, when a user clicks on the business detail page, a new endpoint call to fetch the detailed information of a business is usually required.

APIs for business

The APIs related to a business object are shown in table below.

Table 1.2: APIs for a business

API	Detail
GET /v1/businesses:id	Return detailed information about a business
POST /v1/businesses	Add a business
PUT /v1/businesses/:id	Update details of a business
DELETE /v1/businesses/:id	DELETE a business

If you are interested in real-world for place/business search, two examples are Google Places API [7] and Yelp business endpoints [8]

Data Model

In this section, we discuss the read/write ratio and the schema design. The scalability of the database is covered in deep dive.

Read/Write Ratio

Read volume is high because the following two features are very commonly used.

- Search for nearby businesses
- View the detailed information of a business

On the other hand, the write volume is low because adding, removing, and editing business info are infrequent operations.

For a read-heavy system, a relational database such as MySQL can be a good fit. Let's take a closer look at the schema design.

Data Schema

The key database tables are the business table and the geolocation (geo) index table.

Business Table

The business table contains detailed information about a business. It is shown in Table 1.3 and the primary key is `business_id`.

business	
business_id	PK
address	
city	
state	
country	
latitude	
longitude	

Geo index table

A geo index table is used for the efficient processing of spatial operation. Since this table requires some knowledge about geohash, we will discuss it in the “Scale this database” section on page 24.

High-level design

The high level design diagram is shown in Figure 1.2. The system comprises two parts: location-based service (LBS) and business-related service. Let’s take a look at each component of the system.

Location-based service (LBS)

The LBS service is the core part of the system which finds nearby businesses for a given radius and location.

The LBS has the following characteristics:

- It is a read-heavy service with no write requests
- QPS is high, especially during peak hours in dense areas
- This service is stateless so it’s easy to scale horizontally

Business service

Business service mainly deals with two types of requests:

- Business owners create, update, or delete businesses. Those requests are mainly write operations, and the QPS is not high
- Customers view detailed information about a business. QPS is high during peak hours

Database cluster

- The database cluster can use the primary-secondary setup In this setup, the primary database handles all the write operations, and multiple replicas are used for read operations

- Data is saved to the primary database first and then replicated to replicas
- Due to the replication delay, there might be some discrepancy between data read by the the LBS and the data written to the primary database
- This inconsistency is usually not an issue because business information doesn't need to be updated in real-time

Scalability of business service and LBS

Both the business service and LBS are stateless services, so it's easy to automatically add more servers to accommodate peak traffic (e.g. mealtime) and remove servers during off-peak hours (e.g. sleep time)

If the system operates on the cloud, we can set up different regions and availability zones to further improve availability [9]. We discuss this more in deep dive.

Algorithm to fetch nearby businesses

- In real life, companies might use existing geospatial databases such as Geohash in Redis [10] or Postgres with PostGIS extension [11]
- You are not expected to know the internals of those geospatial databases during an interview iiIt's better to demonstrate your problem-solving skills and technical knowledge by explaining how the geospatial index works, rather than to simply throw out database names
- The next step is to explore different options for fetching nearby business
- We will list a few options, go over the thought process, and discuss trade-offs

Option 1: Two-dimensional search

The most intuitive but naive way to get nearby businesses is to draw a circle with the predefined radius and find all the businesses within the circle as shown in Figure 1.3.

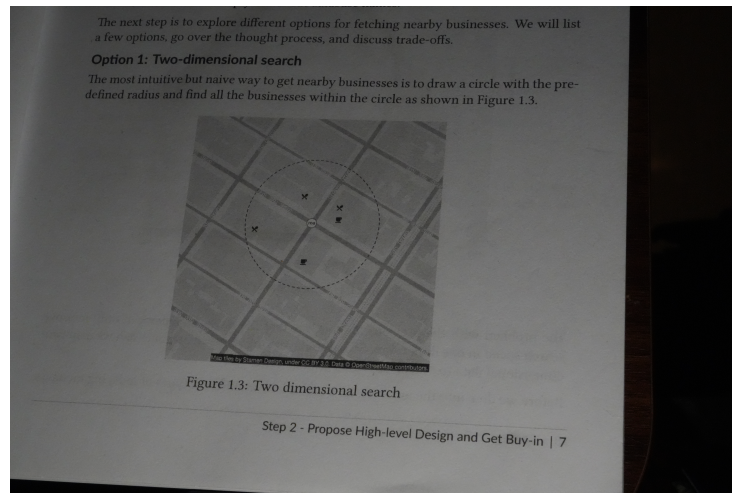


Figure 1.3: Two dimensional search

This process can be translated into the following pseudo SQL query:

```

1 SELECT business.id, latitude, longitude,
2 FROM business
3 WHERE (latitude BETWEEN [:my_lat] - radius AND {:my_lat} +
      radius)
4 AND
5 (longitude BETWEEN {:my_long} - radius AND {:my_long} +
      radius)

```

- This query is not efficient because we need to scan the whole table
- What if we build indexes on longitude and latitude columns?
- Would this improve the efficiency? The answer is not by much
- The problem is that we have two-dimensional data and the dataset returned from each dimension could still be huge
- For example, as shown in Figure 1.4, we can retrieve dataset 1 and dataset 2, thanks to indexes on longitude and latitude columns
- But to fetch businesses within the radius, we need to perform an intersect operation on those two datasets
- This is not efficient because each dataset contains lots of data